

■ ■ ■ AI Audio Forensic Lab (Process 4.0)

System Architecture, Forensic Models, Multi-Phase Fusion, and Integration Specifications

■ # ■ ■ FakeShield AI Audio Forensic Lab — Complete Technical Documentation

This document is the definitive reference for the FakeShield AI Audio Lab. It explains the full audio forensic workflow, model architecture, signal detectors, output invariants, API behavior, and frontend integration.

1. Purpose

The FakeShield AI Audio Lab is designed to analyze uploaded speech and audio files to detect:

- AI-generated voice cloning or TTS
- voice conversion and speaker replacement
- spliced or edited audio segments
- synthetic artifacts from modern neural vocoders

It combines deep learning classifiers, signal processing heuristics, and robustness stress-testing to produce both a verdict and an explanation-friendly forensic report.

2. High-Level Architecture

Components

- **Frontend UI:** React audio lab page, upload controls, progress indicators, and forensic report renderer.
- **API layer:** FastAPI router for asynchronous analysis jobs.
- **Backend analysis pipeline:** audio loading, preprocessing, feature extraction, model scoring, fusion, timeline generation, and explanation.
- **MongoDB persistence:** audit logging of completed scans.

Key file locations

- Backend audio models: `backend/app/models/audio/`
- API router: `backend/app/routers/audio_router.py`
- React frontend page: `fakeshield/src/pages/AudioLab/AudioLabPage.tsx`
- Forensic report display: `fakeshield/src/pages/AudioLab/AudioReportPanel.tsx`
- Upload/poll service: `fakeshield/src/services/audioService.ts`
- Result schema: `fakeshield/src/services/api/audioAnalysis.ts`

3. End-to-End Workflow

3.1 Upload and Validation

The React Audio Lab UI accepts common audio formats:

- WAV, MP3, FLAC, OGG, M4A, MP4

Client-side validation lives in **fakeshield/src/services/audioService.ts**:

- file type by extension
- maximum size 50 MB
- non-empty files

The selected file is uploaded to **POST /audio/analyze/async**.

3.2 Job Creation and Background Processing

backend/app/routers/audio_router.py handles upload submission:

- validates MIME type and extension
- enforces a 50 MB limit
- creates a **job_id**
- runs **analyze_audio** in a background task using **asyncio.to_thread**

A separate status endpoint, **GET /audio/status/{job_id}**, allows the frontend to poll until the analysis is complete.

3.3 Analysis Pipeline

The analysis occurs in **backend/app/models/audio/audio_detector.py**.

Steps:

1. Load audio and normalize.
2. Apply voice activity detection (VAD).
3. Build 5-second audio chunks.
4. Run six signal detectors in parallel.
5. Compute telephony stability.
6. Fuse signal outputs into a final verdict.
7. Build a segment timeline.
8. Generate explainability and recommended actions.
9. Persist scan summary to MongoDB.

4. Data Preprocessing (**backend/app/models/audio/audio_loader.py**)

Responsibilities

- decode uploaded bytes into waveform data
- convert stereo to mono
- resample to 16 kHz
- normalize peak amplitude
- remove silent segments with VAD
- split audio into fixed 5-second chunks

Algorithm details

- Primary decoder: **soundfile** for lossless formats
- Fallback decoder: **librosa.load** for lossy containers via FFmpeg
- Resampling target: **TARGET_SR = 16000**
- Peak normalization: maximum amplitude set to **0.95**
- VAD threshold: **15%** of mean RMS energy
- Chunk length: **CHUNK_SEC = 5.0**
- Minimum chunk duration: **0.5** seconds

5. Signal Detection Modules

FakeShield uses six distinct signals to cover complementary forensic indicators.

5.1 WavLM SSL Deepfake Detection (`backend/app/models/audio/signal_wavlm.py`)

- **Primary model:** `abhishtagatya/wavlm-base-960h-itw-deepfake`
- **Purpose:** detect "in-the-wild" neural speech synthesis
- **Input:** multiple 5-second chunks plus one telephony-resampled chunk
- **Batching:** chunks are processed in a single batched forward pass for efficiency
- **Output:**
 - mean AI probability
 - per-chunk scores
 - model statistics (`max`, `var`)

5.2 AST / Wav2Vec2 Spoof Classification (`backend/app/models/audio/signal_wav2vec.py`)

- **Primary model:** `MattyB95/AST-ASVspoof5-Synthetic-Voice-Detection`
- **Fallback model:** `abhishtagatya/wav2vec2-base-960h-asv19-deepfake`
- **Purpose:** spectrogram-based deepfake and spoof detection
- **Method:** audio is converted into Mel spectrogram features and processed by a transformer
- **Normalization:** outputs are amplified with a power curve to increase contrast in high-confidence ranges

5.3 Speaker Consistency Auditor (`backend/app/models/audio/signal_speaker.py`)

- **Purpose:** detect identity drift or unnatural stability across chunks

- **Technique:**
- tries **pyannote/embedding**
- falls back to **facebook/wav2vec2-base** hidden states
- final fallback uses MFCC mean/std embeddings
- **Features:**
- mean cosine similarity across consecutive chunks
- standard deviation of similarity values
- count of identity jumps
- **Suspicion logic:**
- low mean similarity → voice conversion or splicing
- low similarity variance → overly constant cloned voice

5.4 Prosody and Rhythm Auditor (`backend/app/models/audio/signal_prosody.py`)

- **Purpose:** identify unnatural pitch, timing, and pause patterns
- **Main features extracted:**
- pitch standard deviation in semitones
- pitch range and kurtosis
- IOI coefficient of variation
- pause duration CV
- speaking rate variance
- **Suspect conditions:**
- monotone pitch (**f0_std** very low)
- metronomic rhythm (**ioi_cv** low)
- overly regular pauses
- constant speaking rate

5.5 Spectral Artifact Auditor (`backend/app/models/audio/signal_spectral.py`)

- **Purpose:** detect vocoder and resampling artifacts in the frequency domain
- **Key spectral indicators:**
- MFCC delta variance
- spectral flatness variance
- high-frequency energy ratio
- mel-spectrogram frame-to-frame difference
- spectral centroid variance
- **Global + chunk scoring:** combines a full-waveform global score with per-chunk scores

5.6 Codec and Artifact Auditor (`backend/app/models/audio/signal_codec.py`)

- **Purpose:** catch low-level production artifacts not visible to classifiers
- **Examined artifacts:**
- noise floor and SNR proxy
- DC offset
- spectral ripple from resampling
- ENF power at 50/60 Hz

- near-clipping ratio
- low-pass residual variance (dithering/noise signature)

6. Robustness Stress Test

Telephony stability

- The pipeline resamples the first audio chunk to **8 kHz** and back to **16 kHz**.
- It compares the original WavLM score to the telephony-resampled score.
- The resulting stability metric penalizes large score deltas.
- This catches synthetic audio that changes behavior under real-world channel distortion.

Score formula

```
stability = 1.0 - min(1.0, abs(score_orig - score_telephony) / 0.40)
```

- **>= 0.70**: stable
- **< 0.60**: suspicious

7. Fusion Logic and Verdict Mapping (`backend/app/models/audio/audio_fusion.py`)

Decision hierarchy

1. Strong agreement between WavLM and AST
2. Strong WavLM plus supporting prosody or speaker evidence
3. Speaker/prosody combined inconsistency
4. Telephony instability guard
5. Adaptive weighted blending
6. Authenticity protection if all signals are weak

Weighted blend

- **WavLM**: 40%
- **AST/Wav2Vec**: 20%
- **Prosody**: 15%
- **Speaker**: 15%
- **Spectral**: 5%
- **Codec**: 5%

Final classification

- **AI-Generated**: fused score **>= 0.80**

- **AI-Generated:** fused score ≥ 0.55
- **Suspicious:** fused score ≥ 0.48
- **Authentic:** fused score < 0.48

Threat mapping

- **AI-Generated** → **CRITICAL**
- **Suspicious** → **MEDIUM**
- **Authentic** → **SAFE**

8. Explainability Output (`backend/app/models/audio/audio_explanation.py`)

The project generates a structured explanation object that contains:

- **forensic_summary**
- **recommended_action**
- **confidence**
- **primary_reasons**
- **supporting_reasons**
- **exonerating_factors**
- **stability_report**

Explanation heuristics

- High WavLM scores become critical evidence.
- AST/Wav2Vec support is added when results are strong.
- Prosody evidence is translated into human-readable descriptions like "robotic pitch monotony" or "metronomic speech rhythm".
- Speaker analysis distinguishes between "identity drift" and "unnatural constancy".
- Stability results are included as an exonerating or cautionary note.

9. Timeline Construction (`backend/app/models/audio/audio_segmentation.py`)

A timeline is built from chunk-level scores for visualization:

- Each 5-second interval receives a risk score.
- The score is computed from WavLM/Wav2Vec, spectral, prosody, and speaker signals.
- Timeline segments include start/end timestamps and signal breakdown for UI heatmaps.

Segment output fields

- **segment**
- **start_sec**
- **end_sec**

- `ai_score`
- `level`
- `signals`

10. Frontend Architecture

Audio Lab page

`fakeshield/src/pages/AudioLab/AudioLabPage.tsx` manages:

- file selection and drag-and-drop
- progress and upload states
- analysis start/reset actions
- gauge and status rendering
- result presentation

Report panel

`fakeshield/src/pages/AudioLab/AudioReportPanel.tsx` renders:

- case summary and severity
- fusion rule badge
- action recommendation
- metadata cards
- primary and supporting evidence blocks
- exonerating factors list
- JSON report download

Service layer

`fakeshield/src/services/audioService.ts` implements:

- client-side file validation
- upload to `/audio/analyze/async`
- polling `/audio/status/{job_id}` every 2.5 seconds
- timeout after 180 seconds
- progress messaging for the UI

Result schema

`fakeshield/src/services/api/audioAnalysis.ts` defines the TypeScript shape for results, including:

- signal scores
- timeline segments
- detailed signal metadata
- forensic reasons
- audio metadata

11. Deployment & Operational Notes

API health

GET /audio/health returns available signal names and dependency requirements.

Model loading

- WavLM and AST models are loaded lazily when first needed.
- The system uses **MODEL_LOAD_LOCK** to avoid concurrent model initialization races.
- Models are loaded on CPU and forced into evaluation mode.

Performance

- PyTorch is restricted to a single thread during analysis: **torch.set_num_threads(1)**.
- Parallel signal extraction is limited to 6 workers.
- Audio chunking is capped at the first 3 chunks (15 seconds) for speed.

12. Summary

The FakeShield AI Audio Lab is a mature forensic module that blends:

- deep fake speech classifiers
- spectral and codec artifact heuristics
- speaker identity drift analysis
- prosody and timing analysis
- telephony robustness evaluation
- hierarchical fusion logic
- natural language explainability

This document captures the full lab workflow across backend, API, and frontend implementation.